CHALLENGE 5 · ATL & DFW

Capacity & Risk

We forecast how many aircraft Atlanta and Dallas Fort Worth can land each hour, 3 to 12 hours ahead, and which flights are likely to be delayed by more than 45 minutes. A duty manager sees both on a web dashboard, along with recommendations in plain English.

Branch **codex/vercel-deployment** **19** commits on top of main

Scored on **Jan–Jul 2025**, never used in training ● Live on **Vercel**

WHAT THE BRANCH HOLDS

Raw data in, a live product out

`main` holds the raw flight and weather files and the exploratory notebooks. This branch adds everything that turns them into a working, tested and deployed tool. It has five parts.

Data pipeline

One command downloads public weather observations, official airport forecasts and airline on-time records, then checks and caches them. Each file gets a receipt with its source URL and checksum, so a rerun uses exactly the same data.

`run_pipeline.py` · `stage1_pipeline.py` · `src/download.py` · `src/weather.py` · `src/taf.py` · `src/features.py`

Two models

A gradient-boosted model forecasts the hourly arrival rate for each lead time from 3 to 12 hours. A calibrated logistic regression scores each flight's chance of a delay over 45 minutes. We tested it against LightGBM and it won on 2024 validation data.

`src/train_aar.py` · `src/train_delay.py` · `src/aar.py` · `models/delay_lead3…12.joblib`

Evidence

Every result is saved as a CSV. 70 automated tests, all passing, check that no future data leaks in, that wind is measured along the runway, and that the time splits stay clean. A second pipeline uses the full BTS archive of 3.26 million flights to test forecasts in storm hours.

`reports/*.csv` · `tests/` · `test_stage1_pipeline.py` · `jho/aerocast_data/`

Dashboard

A no-build web page with a 3D model of each real airfield, drawn from OpenStreetMap. It replays the 2025 test period, and its five views go from the next 12 hours down to individual flights.

`dashboard/web/` · `dashboard/server.py` · `src/advisory.py` · `dashboard/build_airfields.py`

Deployment

Vercel serves the page from its CDN and sends every `/api` request to a single Python function. The deployed function runs the same code as the local server. An allowlist controls exactly what gets uploaded, and nothing is trained at deploy time.

`vercel.json` · `.vercelignore` · `api/index.py` · `.python-version` · `tests/test_vercel.py`

HOW A FORECAST IS MADE

From public data to a recommendation

INPUTS
Public data
Weather reports (METAR), airport forecasts (TAF), BTS flight records

→

FEATURES
No look-ahead
Joined by the time each report was published, with wind measured along the runway

→

CAPACITY
Arrival rate
Arrivals per hour, with an 80% range, for each lead from 3 to 12 h

→

RISK
Severe delay
Calibrated probability per flight, plus the factors driving it

→

OUTPUT
Advisory
Simple, transparent rules flag reduced-capacity windows. The duty manager makes the call.

RESULTS · TEST PERIOD JAN–JUL 2025

Clearly better than the baselines

32%

lower arrival-rate error than persistence, 3 hours ahead

`RMSE 12.2 vs 17.9 /h`

45%

lower error than persistence, 12 hours ahead

`RMSE 12.9 vs 23.3 /h`

2.8×

the normal severe-delay rate among the 10% of flights rated riskiest

`37.5% vs 13.2% base rate, 3 h ahead`

ARRIVAL-RATE ERROR (RMSE, ARRIVALS/H)	+3 H	+6 H	+12 H
Our model	**12.2**	**12.7**	**12.9**
The airport forecast (TAF) put through our capacity rule	15.4	15.3	15.5
Climatology (usual rate for that month and hour)	18.4	18.4	18.4
Persistence (assume nothing changes)	17.9	20.7	23.3

SEVERE-DELAY RANKING (PR-AUC, HIGHER IS BETTER)	+3 H	+6 H	+12 H
Our model	**0.319**	**0.264**	**0.269**
Random guess (equals the base rate)	0.132	0.132	0.132

Sources: `reports/aar_metrics.csv`, `reports/delay_metrics.csv`. The model's 80% range contains the real value 88–89% of the time, which errs on the safe side.

THE LIVE DASHBOARD

Five views, from the big picture to single flights

Outlook — The 3D airfield, a 12-hour capacity strip, a delay radar and the recommendation. Choose Balanced or Cautious alerting. Shortcut chips jump to real 2025 capacity drops.

Flights — Risk for every sampled flight, with its main drivers. Turn on reveal to compare each one with what actually happened.

History — Reduced-capacity hours by day, plus a weekday-by-hour heatmap of when the airport usually loses capacity.

Evaluation — Accuracy for each lead time, and a reliability chart that checks whether a predicted 30% risk really happens about 30% of the time.

Storms — How the forecasts held up during NOAA-logged storm hours. NOAA logs only label the hours; they are never a model input.

Optional: with `ANTHROPIC_API_KEY` set, Claude writes a plain-English summary of the outlook. Without the key, that button stays hidden and everything else still works.

HOW IT RUNS ON VERCEL

A static page and one Python function

`Browser → Vercel CDN dashboard/web · /api/* → api/index.py → dashboard/server.py → committed replay snapshot`

Frontend

`dashboard/web/`

- Plain HTML, CSS and JavaScript with no build step
- three.js loads from a pinned CDN version
- Real runways and terminals baked in from OpenStreetMap

API

`api/index.py → dashboard/server.py`

- Six JSON routes: meta, outlook, history, eval, storm and explain
- Uses the same handler locally and on Vercel
- Python 3.13, 60-second time limit

What ships

`.vercelignore + vercel.json`

- 4 processed data files, the airport lookup and 10 delay models
- Raw downloads, notebooks and training code are excluded
- Checked against Vercel's 500 MB function limit

What it is not

`historical replay`

- It replays January–July 2025 and is not a live feed
- The pipeline runs locally; its outputs are committed to the repo
- `tests/test_vercel.py` runs the deployed handler against the snapshot

DESIGN CHOICES

What the judges are likely to probe

Leakage — Every input must have been published before the forecast time. TAFs must be issued at least the lead time before the target hour, and an expired TAF counts as missing. Tests fail if anything breaks these rules.

Runway direction	Wind is split into headwind, crosswind and tailwind using each runway's heading (ATL 80°, DFW 180°). Crosswind and tailwind cut capacity, while raw wind speed does not.
Not tied to one airport	The target is a share of each airport's best-case rate, and airport ID is not a feature. An unseen-airport test is reported.
Fair delay comparison	Delay is the share of flights over 45 minutes, never raw minutes, and results are reported per airport and per direction.
Honest split	Train on Jan 2023–Jun 2024, validate on Jul–Dec 2024, test on Jan–Jul 2025. Data is never shuffled, and flights that cross a split boundary are removed.

LIMITS

Raise these before the judges do

Capacity "truth"	No public dataset logs the hourly arrival rate airports actually set. Our target comes from FAA-published rates by weather category. Real BTS arrivals move in the same direction as that rule, but its absolute level is not confirmed.
Storms	Errors roughly double in storm hours (about 17–20 arrivals/h vs 7–8), and the 90% range contains the real value only about half the time. Treat a storm-hour forecast as a warning, not an exact count.
New airports	The pipeline handles any airport code. A model trained only on ATL does worse on DFW than DFW's own airport forecast.
Sample size	The organisers' flight file is a small sample, so the dashboard shows risk rates rather than flight counts.
Not live	The deployed site replays the test period. A live feed is the next step.

SUGGESTED WALKTHROUGH

Three minutes in front of the judges

1

0:00–0:30

The problem.

Storms cut how many aircraft an airport can land, and today the cut usually comes after the weather arrives.

2

0:30–1:30

Outlook view, live.

Click an event chip to reload the outlook six hours before a real capacity drop. Show the capacity strip falling, the 3D airfield and the recommendation.

3

1:30–2:00

Flights view.

Pick a high-risk flight, show its drivers, then turn on reveal to show the actual delay.

4

2:00–2:40

Evaluation and Storms.

32–45% lower error than persistence and a calibrated delay risk, plus where the model struggles.

5

2:40–3:00

Under the hood.

No look-ahead, enforced by tests. One command rebuilds everything. The deployment is one static page and one function.